

BEYOND THE VIBE

Context Engineering

Usando context engineering para llevar
flujos agénticos a producción

Bryan Condor

Software Engineer @ Addi · Meetup · Mayo 2026

Vibe coding funciona

3 razones por las que es real

Velocidad

De idea a demo en una tarde. A veces en horas.

Low barrier

Cero curva de aprendizaje. Empiezas y avanzas.

Exploración

Ideal para validar una idea antes de invertir tiempo serio.

Hasta acá llega

Greenfield brilla. Brownfield rompe.

Greenfield ✓

Idea → demo en horas.

Sin lineamientos previos, vibe coding alcanza.

Brownfield ✗

Tu proyecto productivo tiene lineamientos. Dos trampas:

- Repites TODO cada iteración
- O lo dejas avanzar y se sale del carril

No fue el modelo.

Fue el contexto que le di.

*"Most agent failures are not model failures anymore, they are
context failures."*

– Phil Schmid

2025

Cuatro formas en que tu contexto falla



POISONING

La alucinación que se autoreferencia sesión tras sesión



DISTRACTION

Demasiado contexto: el modelo se pierde en él



CONFUSION

El "por si acaso" que degrada la respuesta

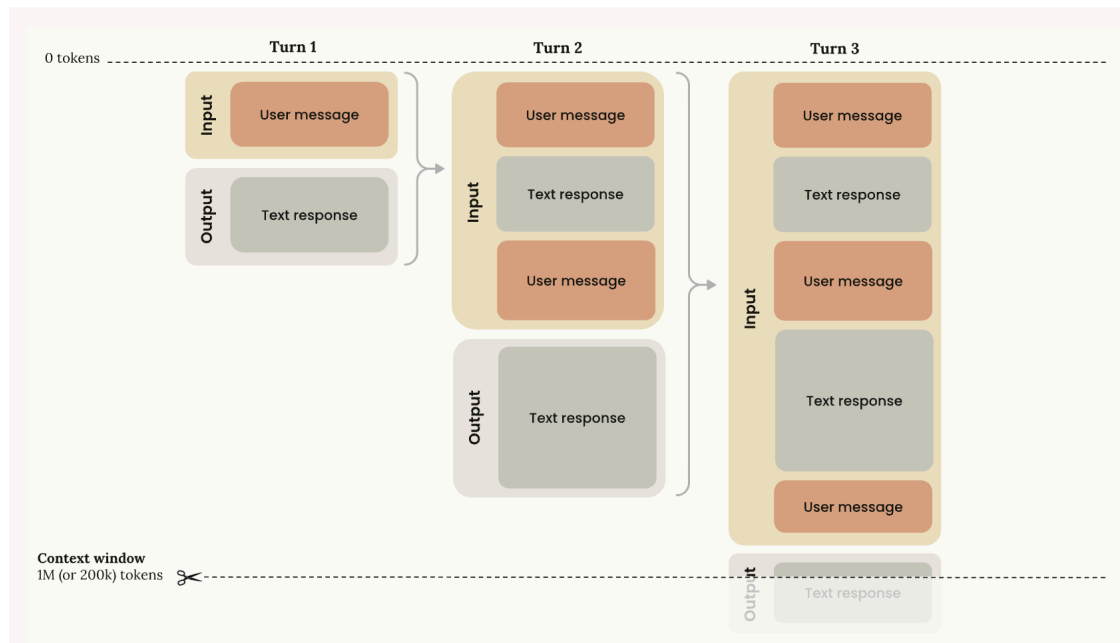


CLASH

Tu spec dice X, tu código hace Y

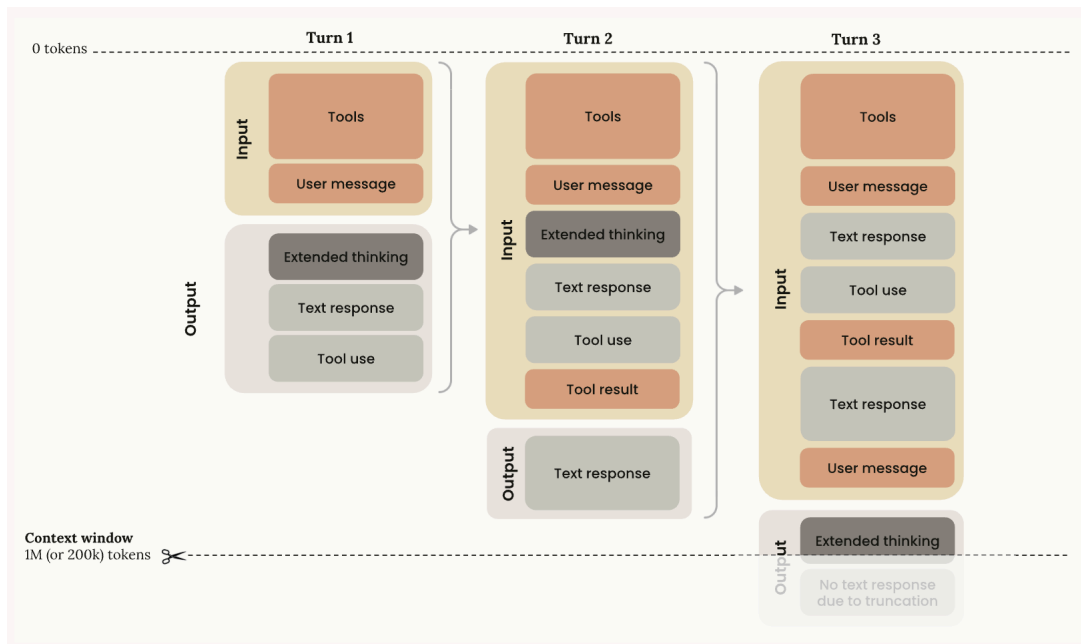
**¿Cuántos de estos
han visto en sus
propios agentes?**

¿Qué hay en el context-window?



Cada turn acumula el anterior. Solo con user message + response, los tokens ya suben.

¿Y si agregas tools?



Tools, tool results, extended thinking... en 3 turns ya se trunca.

Dos definiciones

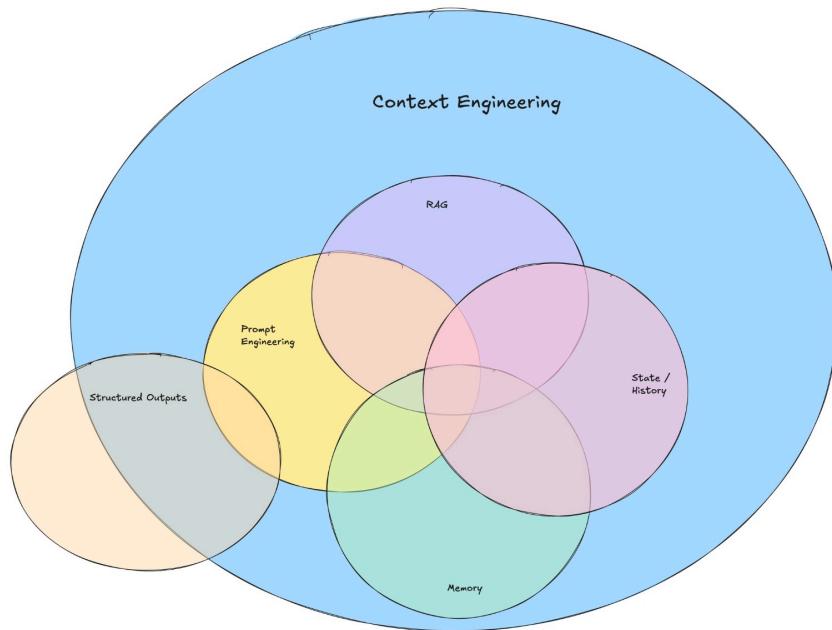
*"Context engineering is the **delicate art and science** of filling the context window with just the right information for the next step."*

– Andrej Karpathy · 2025-06-25 · *(poética)*

*"Context engineering is **building dynamic systems** to provide the right information and tools in the right format such that the LLM can plausibly accomplish the task."*

– Harrison Chase · LangChain · 2025-06-23 · *(operacional)*

Lo que involucra Context Engineering



Source: [@dexhorthy](#)

Dato curioso: el término "Context Engineering" se acuñó entre junio y septiembre de 2025 (Tobi Lütke → Karpathy → Anthropic).

Prompt Engineering $\neq$ Context Engineering

	Prompt Engineering	Context Engineering
Unit of work	A string	A system
Time scope	Single call	Multi-turn, long-horizon
Components	Words, phrases	Instructions + RAG + state + memory + tools
Skill	Writing	Engineering
Typical failure	"Bad prompt"	Poisoning / Distraction / Confusion / Clash

*"Prompt engineering is a **subset** of context engineering."*

– Harrison Chase

Dos estándares emergentes

AGENTS.md

[agents.md](#)

El **entry point** del agente.

Se carga al inicio de cada sesión.

SKILL.md

[agentskills.io](#)

Procedural how-to.

Se carga on-demand cuando se invoca.

AGENTS.md

agents.md standard

*"Instead of treating AGENTS.md as the **encyclopedia**, we treat it as the **table of contents**."*

– Ryan Lopopolo · OpenAI Codex case study

Se carga **al inicio** de cada sesión. Todo el documento. Cada vez.

SKILL.md

agentskills.io standard · developed by Anthropic

*"Full instructions load **only when a task calls for them**, so agents can keep many skills on hand with only a **small context footprint**."*

– **agentskills.io** · spec oficial

Se carga **on-demand** cuando el agente invoca la skill o tipeas `/skill-name` .

Memory Bank existe.

No lo usamos.

Patrón community para Cline. Lo que cubre, ya lo cubrimos con AGENTS.md + SKILL.md.

cline.bot

Sin context engineering

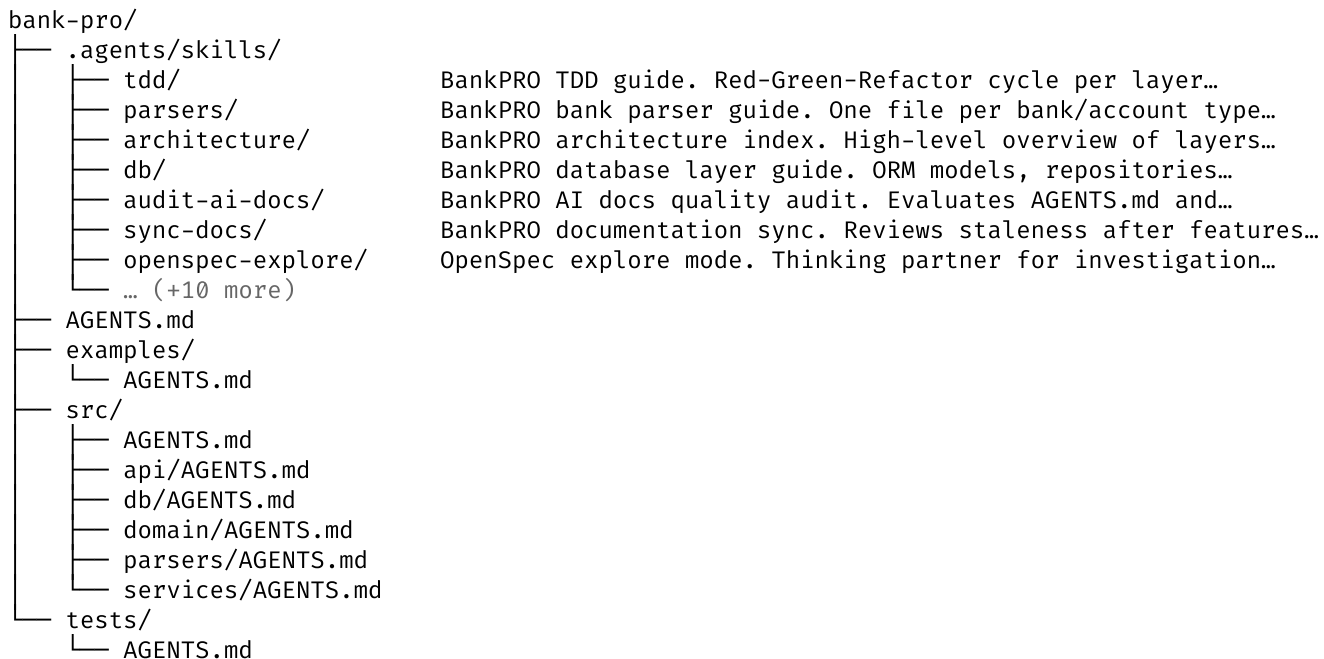
Cómo se vería bank-pro sin AGENTS.md ni SKILL.md

```
bank-pro/
├── src/
│   ├── api/
│   ├── db/
│   ├── domain/
│   ├── parsers/
│   └── services/
├── tests/
├── scripts/
├── docker-compose.yml
├── pyproject.toml
├── README.md
└── uv.lock
```

Código puro. Cero context engineering. ¿Cómo le explicas a un agente nuevo qué hace este proyecto?

Solo context engineering

Output de `context-engineering-map ~/bcd/project/bank-pro`



Juntas

Código + Context Engineering conviviendo

```
bank-pro/
├── AGENTS.md
├── .agents/skills/ ← 17 SKILL.md
├── src/
│   ├── AGENTS.md
│   ├── api/
│   │   ├── AGENTS.md
│   │   └── ... (.py files)
│   ├── db/
│   │   ├── AGENTS.md
│   │   └── ... (.py files)
│   ├── domain/
│   │   ├── AGENTS.md
│   │   └── ... (.py files)
│   ├── parsers/
│   │   ├── AGENTS.md
│   │   └── ... (.py files)
│   └── services/
│       ├── AGENTS.md
│       └── ... (.py files)
├── tests/
│   ├── AGENTS.md
│   └── ... (test files)
├── docker-compose.yml
├── pyproject.toml
└── README.md
```

LEYENDA

■ AGENTS.md

■ .agents/skills/

■ Código + configs

AGENTS.md por dentro

Estructura del AGENTS.md raíz de bank-pro (condensado para el slide)

```
# BankPRO – AI Agent Context

> Read this first. Navigate downstream...

## What Is BankPRO          ...

## Architecture – DDD + Flat Hexagonal
  ### The One Rule
  ### Dependency Table
  ### Architecture Rules
    1. domain/ pure Python, no I/O
    2. services/ orchestrates via ports
    3. Protocols in services/ports.py
    4. api/ routes ≤ 20 lines
    5. One parser per bank × type
    6. New bank = one new file
    7. Never float for money

## Tech Stack                ...
## Module Structure          ...
## Development Workflow      ...
```

9 AGENTS.md en bank-pro

Inventario completo – uno por subdominio

`AGENTS.md` – architecture, rules, navigation

`src/AGENTS.md` – layer dependency graph

`src/api/AGENTS.md` – FastAPI route patterns

`src/db/AGENTS.md` – SQLAlchemy + Alembic

`src/domain/AGENTS.md` – pure Python value objects

`src/parsers/AGENTS.md` – one file per bank

`src/services/AGENTS.md` – ports + orchestration

`tests/AGENTS.md` – pytest, fixtures, conventions

`examples/AGENTS.md` – sample integration specs

SKILL.md por dentro

Estructura del SKILL.md de `api` (condensado)

```
name: api
description: BankPRO REST API guide ...
allowed-tools: Read Glob Grep

# BankPRO REST API
> Location: src/api/ • Inbound adapter • Max ~20 lines/route

## Files
src/api/
├─ main.py      ← FastAPI app factory
├─ routes.py    ← all route handlers
├─ schemas.py   ← Pydantic models

## Route Pattern
  validate → call service → serialize

## Pydantic Schemas      ...
## Dependency Injection  ...
## Error Handling        ...
## Common Mistakes       ...
## What to read next     ...
```

17 SKILL.md en bank-pro

Inventario completo – procedural how-tos por layer y workflow

[tdd/SKILL.md](#) – Red-Green-Refactor cycle

[parsers/SKILL.md](#) – bank parser impl

[architecture/SKILL.md](#) – layers overview

[db/SKILL.md](#) – ORM + alembic

[api/SKILL.md](#) – FastAPI + Pydantic

[domain/SKILL.md](#) – value objects

[services/SKILL.md](#) – ports + orchestration

[testing/SKILL.md](#) – pytest patterns

[tech-stack/SKILL.md](#) – libraries + configs

[start/SKILL.md](#) – feature onboarding flow

[sync-docs/SKILL.md](#) – doc staleness check

[audit-ai-docs/SKILL.md](#) – quality auditor

[openspec-explore/SKILL.md](#) – SDD explore

[openspec-propose/SKILL.md](#) – SDD propose

[openspec-apply-change/SKILL.md](#) – SDD apply

[openspec-archive-change/SKILL.md](#) – SDD archive

[local-env/SKILL.md](#) – setup + docker + uv

Lecciones

Tres cosas que cambiaron cómo trabajo · 1 de 3

Sigue el **estándar** desde el día 1.

EL ESTÁNDAR

```
AGENTS.md  
.agents/skills/
```

Define una vez. Todos los CLIs lo respetan.

ADAPTACIONES POR HERRAMIENTA (SYMLINKS)

```
CLAUDE.md      → AGENTS.md  
.claude/skills/ → .agents/skills/  
.cursor/rules/  → .agents/skills/
```

Cambia la herramienta, no el estándar.

Lecciones

Tres cosas que cambiaron cómo trabajo · 2 de 3

Cada error del agente, arreglado en `AGENTS.md`, sobrevive **forever.**

Compounding. La memoria del proyecto crece sin que la pidas.

Lecciones

Tres cosas que cambiaron cómo trabajo · 3 de 3

Las skills **auditan** a las skills.

`/audit-ai-docs`

Puntúa AGENTS.md + SKILL.md contra rúbrica de calidad.
Surface gaps.

`/sync-docs`

Detecta staleness después de cada feature. Avisa si la doc se
desfasa del código.

La disciplina se vuelve **automática**.

Vibe coding para **empezar**. Context engineering para **escalar**.



Bryan Condor

Software Engineer @ Addi

linkedin.com/in/bryancondor



Feedback de la charla (60 seg)

3 disciplinas

Dónde estamos · dónde vamos

Prompt Engineering

Unit: un string

Scope: 1 call

Falla: "mal prompt"

Context Engineering

Unit: context window

Scope: sesión multi-turn

Falla: 4 failure modes

Harness Engineering

Unit: entorno completo

Scope: multi-sesión long-horizon

Falla: agent drift, slop accumulation

Cada disciplina **contiene** a la anterior.

– Addy Osmani · 2026

Próximo nivel: harness engineering

*"Agent = Model + Harness.
If you're not the model, you're the **harness**."*

– Addy Osmani

*"The most productive AI teams in 2026 spend more
time building their **harness** than writing code."*

– Addi Engineering

Si esto les hizo sentido, el siguiente paso es:

sub-agents · MCP servers complejos · evals automatizados · CI loops para agentes · cloud agent infrastructure

Eso es harness engineering. Charla para otro meetup.